Découverte du Système d'exploitation Linux

Njakarison Menja R.

Découverte du système GNU/Linux

- ***Architecture générale du système*** :
  - système d'exploitation multi-tâche multi-utilisateurs
- ***Le noyau UNIX*** :
  - programme qui assure la gestion de la mémoire, le partage du processeur entre les différentes tâches à exécuter et les entrées/sorties de bas niveau.
  - d'assurer une bonne répartition des ressources de l'ordinateur (mémoire, processeur(s), espace disque, imprimante(s), accès réseaux) sans intervention des utilisateurs.

INTERPRÉTEURS DE COMMANDES

- ***L'invite du shell :***
  - Avec windows : C:> D:> E:\dos ...
  - Avec linux :
    - [root@local /etc]# : utilisateur root, machine local, le répertoire courant, est /etc et le # indique que c'est l'administrateur (super user).
    - [lobo@solosainako /home]$: utilisateur lobo, machine solosainako, et le répertoire courant, est /home # indique que c'est un simple utilisateur
- ***L'interpréteur de commandes*** : est un logiciel faisant partie du système d'exploitation Linux dont le but est de faire l'interface entre la personne travaillant sur l'ordinateur et les commandes qui sont exécutée.
- Il y en a plusieurs, les plus utilisés sont : bash, dash, zsh, csh, tcsh, ksh,...
- sh est le shell historique depuis le système unix 7. Ses fonctions de base ont été étendues avec les interpréteur ash puis bash. bash : l'interpreteur GNU, dash : l'interpréteur de Debian, zsh : plus léger que bash et plus étendu

COMMANDES DE BASE

- ***structure d'une commande linux*** : commande option argument
  - Ex : ls -a /etc :
    - ls : commande pour lister le contenu d'un repertoire
    - -a : option qui permet d'inclure les fichiers cachés
    - /etc : argument, repertoire cible

  pour afficher le manuel d'une comande :
  - man <commande> (ex : man ls)
- ***Commande de base***
  - ls - Lister contenu du répertoire courant
  - more - Afficher le contenu d'un fichier
  - cat - Concatener des fichiers
  - cp - Copier un fichier
  - mv - Déplacer et renommer un fichier
  - rm - Détruire un fichier
  - find - Chercher un fichier
  - grep - Recherche dans un fichier
  - cd - Changement de répertoire
  - mkdir - Création de répertoire
  - rmdir - Destruction de répertoire
  - pwd - connaître le répertoire courant
  - clear - Efface l'écran
  - df -Espace disque
  - ps - lister les processus
  - kill - Tuer un processus
  - diff - Comparer deux fichiers
  - tar - Création d'archive
  - ...

Gestion des utilisateurs sous GNU/Linux

- Notion d'utilisateurs et groupes
- Gestion des utilisateurs et des groupes

Notion d'utilisateurs et groupes

- ***Utilisateurs :***
  - Le système, dès son installation, avant même la première connexion au système a créé des users système
  -
- ***Les principales commandes***

useradd, usermod, userdel	gestion des comptes utilisateur
groupadd, groupmod, groupdel	gestion des groupes
pwck, grpck	vérification des fichiers
passwd	changer le mot de passe d'un utilisateur
chfn, id, groups, finger	utilitaires divers

Notion d'utilisateurs et groupes

- ***Créer, modifier et supprimer un utilisateur :***
  - Il faut être superutilisateur (root)
  - Commande : useradd rakoto
    - Crée un utilisateur avec l'identifiant rakoto et un uid (user ID) unique
    - le répertoire personnel /home/rakoto (par defaut le nom du compte)
    - une nouvelle entrée dans les 2 fichiers fondamentaux /etc/passwd et /etc/group
  - Pour lui attribuer le mot de passe : passwd rakoto (puis saisir le mot de passe 2 fois)
  - Pour supprimer le compte : userdel rakoto
  - Pour modifier le compte : usermod [option] rakoto
    - Ex : usemod -G groupe1, groupe2 rakoto

      => ajouter rakoto dans les 2 groupes groupe1 et groupe2 qui existent déjà

Notion d'utilisateurs et groupes

- ***Groupes*** :
  - ensemble d'utilisateurs qui possèdent les mêmes droits sur des fichiers/repertoires/commandes ..
  - Chaque utilisateur doit faire partie au moins d'un groupe
  - dans /etc/passwd chaque utilisateur possède un groupe par défaut, précisé par son identifiant gid (groupe ID)
  - Tout utilisateur peut faire partie de plusieurs groupes autres que son groupe primaire
  - Pour lister tous les groupes : groups rakoto
  - Pour créer un nouveau groupe : groupadd groupe3
  - Supprimer un groupe: groupdel groupe3
  - Pour ajouter un utilisateur à un groupe: éditer le fichier /etc/group et d'ajouter les noms d'utilisateurs (séparés par des virgules) sur la ligne du groupe

Les fichiers utiles

- Tout ce qui concerne la gestion et l'authentification des utilisateurs est inscrit dans un seul fichier /etc/passwd
- La gestion des groupes est assurée par /etc/group
- Les mots de passe cryptés sont maintenant placés dans /etc/shadow,par sécurité lisible seulement par root.

Structure du fichier /etc/passwd

Fichier comprend 7 champs, séparés par un “:”

1) nom de connexion

2) ancienne place du mot de passe crypté

3) numéro d'utilisateur uid, sa valeur est le véritable identifiant pour le système Linux; l'uid de root est 0, le système attribut conventionnellement un uid à partir de 500 aux comptes créés.

4) numéro de groupe gid, dans lequel se touve l'utilisateur par défaut;le gid de root est 0, des groupes d'utilisateurs au delà de 500

5) nom complet, il peut être suivi d'une liste de renseignementspersonnels (cf chfn)

6) rép. personnel (c'est également le rép. de connexion)

7) shell, interprétateur de commandes (par défaut /bin/bash)

Structure du fichier /etc/group

fichier comprends 4 champs, séparés par un “:”

1) nom du groupe

2) x pour remplacer un mot de passe non attribué maintenant

3) numéro de groupe, ou group ID (gid)

4) la liste des membres du groupe

La structure d'une ligne de /etc/passwd et de /etc/group

/etc/passwd

- `login:x:uid:gid:commentaires:home:shell`

/etc/group

- `groupe:x:gid:liste-groupes-secondaires`

Options pour useradd

Syntaxe : `useradd [options] nom_login`

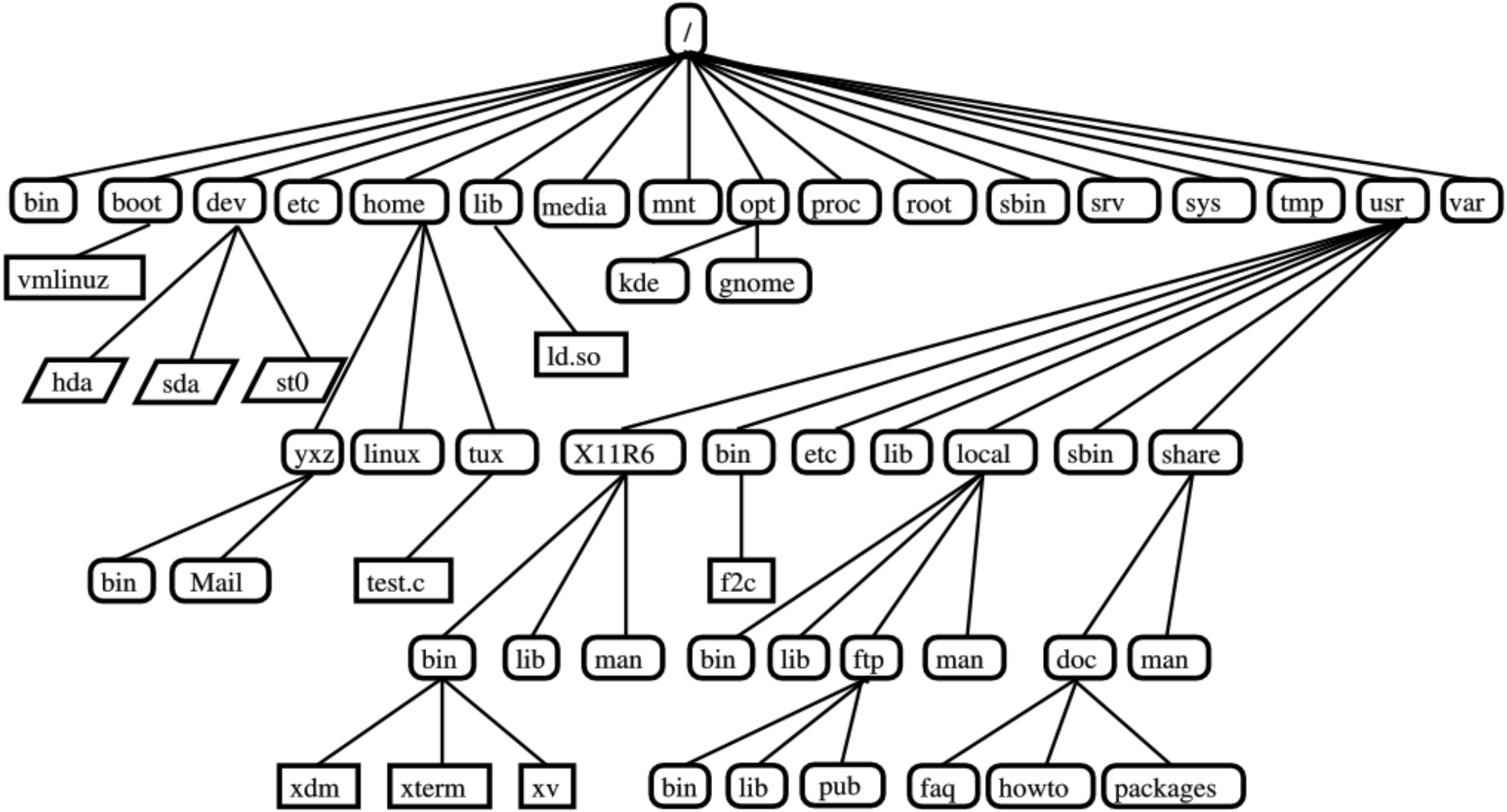
Exemple : `useradd toto -u 1200 -p moi -g 520 -G groupes -s /bin/bash`

-u uid	pour fixer l'identifiant uid
-g groupe-primaire	
-G liste	fixe l'appartenance de l'utilisateur à une liste de groupes secondaires (séparateur , sans espace)
-s shell	par défaut, attribution du shell par défaut bash
-c commentaire	
-d rep. personnel	par défaut dans le répertoire /home
-e date-expiration	fixe la date d'expiration du compte (format MM/JJ/AA)
-m	pour créer le répertoire personnel
-k rep-skel	recopie le contenu de rep-skel dans le rép. personnel, par défaut /etc/skel

Gestion des fichiers

Les principaux répertoires du système

- La figure représente l'arborescence des répertoires sous Linux



Les principaux répertoires du système

Parmi ces répertoires, plusieurs sont important

Répertoire	Contient
/bin	les fichiers exécutables nécessaires à l'initialisation
/boot	le noyau et les fichiers de démarrage
/dev	les fichiers spéciaux
/etc	les fichiers de configuration du système et certains scripts
/home	la base des répertoires utilisateurs
/lib	les librairies système et les modules
/lost+found	le stockage des fichiers retrouvés par fsck
/mnt	les points d'ancrage des systèmes extérieurs
/proc	un système de fichiers virtuels permettant l'accès aux variables du noyau
/root	le répertoire de base du super utilisateur
/sbin	les fichiers exécutables pour l'administration du système
/tmp	les fichiers temporaires
/usr	les programmes, les librairies et les fichiers accessibles pour l'utilisateur
/var	les données variables liées à la machine (spool, traces)

Le répertoire de base / s'appelle : répertoire racine (root)

Types de fichier et permissions d'accès

Les différents types de fichiers sous Linux sont :

- les fichiers normaux : ce sont des collections d'octets. Il n'existe pas de différence entre les fichiers texte et les fichiers binaires.
- les répertoires : ce sont des fichiers contenant les noms des fichiers et leurs numéros d'inode.
- les liens symboliques : permettent de présenter une image d'un fichier sous un autre nom ou à un autre endroit sans dupliquer les données.
- les fichiers spéciaux en mode bloc : sont les portes sur les périphériques fonctionnant par blocs de données (ex : disques).
- les fichiers spéciaux en mode caractère : sont les portes vers les périphériques fournissant ou consommant les données octet par octet.
- les tubes nommés "fifo" : permettent à deux processus sans relation de parenté de s'échanger des données comme par un tube.

Découverte du système GNU/Linux

Pour afficher les attributs principaux des fichiers : ls -l

```
-rw-r--r--      2       root    root    6656    Apr 15  1998    fichier
prw-r--r--      1       root    root      0     Apr 15  1998    fifo
brw-r--r--      1       root    root      0     Apr 15  1998    bloc
crw-r--r--      1       root    root      0     Apr 15  1998  caracteres
drwxr-xr--      1       root    root    1024    Nov 12 19:42 répertoire
```

- le premier caractère donne le type du fichier :

  `-` fichier normal, `p' pour un fifo, `b' pour un fichier spécial en mode bloc, `c' pour un fichier spécial en mode caractère, `d' pour un répertoire

- les neuf caractères suivants donnent les droits d'accès
- le champ suivant donne le nombre de liens sur le fichier
- le champ suivant le nom du propriétaire et du groupe du fichier
- le champ suivant donne la taille en octets du fichier
- la date de la dernière modification

La commande stat permet d'afficher plus d'informations sur un fichier

Permissions d'accès aux fichiers

Tout fichier appartient à un utilisateur (son "propriétaire") et à un groupe.

Pour chaque fichier le monde de ses utilisateurs est scindé en 3 catégories :

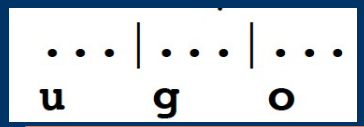
- u, l'utilisateur normal, son propriétaire (son créateur)
- g, son groupe, ensemble d'utilisateurs ayant parfois des "permissions" particulières.
- o, tous les (others) autres.

Linux permet de spécifier les droits d'action sur un fichier, que peuvent exercer les utilisateurs des 3 catégories précédentes.

- “r”, lecture autorisé, “-” si non
- “w”, écriture autorisé, “-” si non
- “x”, exécution autorisé, “-” si non

Ex:

```
$ ls -l /dev
$ drwxr-xr-x  3 root  root  60 Feb 16 05:52 bus
```



Permissions des répertoires

Pour répertoires, la signification des attributs est différente de celle d'un fichier normal mais toujours identique pour les 3 catégories d'utilisateurs du répertoire.

- “r”, lire le contenu autorisé, “-” si non (avec ls ou dir)
- “w”, modifier le contenu autorisé, “-” si non (avec cp, mv, rm)
- “x”, accéder aux fichiers du répertoire autorisé, “-” si non

Remarque :

L'utilisateur qui crée un fichier en devient le propriétaire, et le groupe auquel l'utilisateur appartient (au moment de la création) devient le groupe du fichier.

Changer le propriétaire ou le groupe

- ***Changer le propriétaire*** : `chown [-R] utilisateur fichiers`

Réservée au propriétaire actuel des fichiers ou des répertoires (et à root)
L'option -R (récursif) permet d'agir sur l'ensemble des sous répertoires.

Exemple : `chown -R rakoto /home/rabekoto`

- ***Changer le groupe propriétaire*** : `chgrp [-R] groupeX fichiers`

Ceci doit être effectué par root ou le propriétaire, à condition que celui-ci soit membre de groupeX.

Exemple : `chgrp -R groupe4 /home/rakotobe`

- ***Changer les 2 en même temps*** : `chown rakoto.groupe5 fichiers`

Dans ce cas, en plus, le groupe propriétaire des fichiers est changé pour le groupe primaire du nouveau propriétaire

Permissions sur les fichiers

- Les droits d'accès peuvent être modifiés par le propriétaire des fichiers ou par root (ou équivalent, d'uid 0)
- La commande chmod (change mode, change le "mode" des fichiers) peut s'écrire de plusieurs façons équivalentes, sur le modèle :

  ```
  $ chmod droits fichiers.
  ```

  Le paramètre droits permet de calculer les nouveaux droits d'accès.
- Changement de façon relative,
  - par ajout (symbole +) ou
  - retrait (-) par rapport aux droits existants,
  - ou bien de façon absolue, en fixant les nouveaux droits qui remplacent les anciens (symbole =)

Ajout, retrait, fixation des permissions

Pour chaque fichier, on désigne par :

- u, g et o les 3 catégories d'utilisateurs (user, group, other) et de plus par a (=all) tous les utilisateurs.
- r,w,x les 3 attributs de chaque fichier, pour chaque catégoried'utilisateur.
- + - = l'action d'ajouter, de retirer ou de fixer un droit, qui s'applique à chaque catégorie séparément.
- les changements, sur le modèle "à quelle(s) catégorie(s), quelle action, quel(s) droit(s)" sont alors notés symboliquement :

  ```
  [u g o a] [+ - =] [r w x]
  ```

- par exemple

  ```
  #chmod u+x fichier
  ```

  signifie "ajouter le droit d'exécution au propriétaire du fichier

Ajout, retrait, fixation des permissions

- on peut regrouper les catégories si on veut exercer la même action :
  - `#chmod ug+w fichier` => "ajouter le droit d'exécution au propriétaire et au groupe"
  - `#chmod go-rwx` => fichier "enlever tous droits d'accès à tous les utilisateurs, sauf au propriétaire"

Ajout, retrait, fixation des permissions

- ***Notation relative*** : `chmod [-R] <action-droits> fichiers`
  - L'option -R (récursif) modifie les permissions des sous-répertoires.
  - exemple : `chmod [-R] go-rwx /home/rakoto` enlève tous les permissions d'accès des fichiers du rép. personnel de rakoto (et des sous-rép.), à tous sauf au propriétaire, c'est-à-dire rakoto.
- ***Notation absolue***
  - permet de fixer les nouveaux droits qui remplacent les anciens. Si une catégorie n'est pas présente, ses anciens droits s'appliquent.
  - `chmod u=rwx,g=rw,o=r fichiers` => remplace les permissions précédentes des fichiers, en les fixant à `"-rwxrw-r--"`
  - `chmod u=rwx,g=r fichiers =>` fixe les permissions à `-rwxr--???` et ne pas les permissions précédentes du groupe other
  - `chmod u=rwx,g=r,o= fichiers =>` fixe les permissions à `-rwxr-----`

Ajout, retrait, fixation des permissions

Notation octale des permissions :

c'est la plus simple façon d'indiquer les permissions de chaque catégorie.

la table de correspondance

```
Binaire ---- Droit ------ Octal
000 -------- (---) ------- 0
001 -------- (--x) ------- 1
010 -------- (-w-) ------- 2
011 -------- (-wx) ------- 3
100 -------- (r--) ------- 4
101 -------- (r-x) ------- 5
110 -------- (rw-) ------- 6
111 -------- (rwx) ------- 7
```

exemple :

`chmod 764 test => chmod u=rwx,g=rw,o=r test`

Le masque de protection umask

- à la création d'un fichier ou d'un répertoire
  - son propriétaire est l'utilisateur qui l'a créé
  - son groupe est le groupe primaire de ce même utilisateur
  - Les permissions maximales accordées par un fichier et un répertoire sont 666 (-rw-rw-rw-) et 777 (-rwxrwxrwx)
- La commande umask fixe les permissions masquées (les droits non accordés aux fichiers et répertoires lors de leur création)
- Exemple : permissions effectives, affectées lors de la création d'un répertoire, par un utilisateur dont le masque de protection est 027
  - 777 = 111 111 111 permissions maxi = rwx rwx rwx
  - - 027 = 000 010 111 masque de protection
  - = 750 = 111 101 000 permissions effectives = rwx r-x ---

La commande umask

- `umask` => affiche le masque de l'utilisateur actif
- `umask -S` => affiche les permissions correspondantes au masque, sous forme symbolique.
- `umask masque` => fixe les permissions ultérieures de création desfichiers de l'utilisateur actif, conformément à masque, en notation octale. Attention ! le changement ne s'applique qu'à la présente session.
- Pour la rendre permanente, on peut intervenir sur un fichier profile
  - Dans le fichier profil général /etc/profile, on peut modifier la règle habituelle :

```
if [ $UID == 0 ] ; then umask 022 ; else umask 077 ; fi
```

Droits étendus : SUID, GUID, Sticky Bit

- ***SUID*** : permet à un fichier exécutable de s'exécuter sous les droits de son propriétaire, à la place de l'utilisateur qui l'exécute
  - symboliquement s à la place du x du propriétaire u (octale 4000)
    Ex:

```
$ ls -l /etc/passwd
$ sudo chmod u+s /etc/passwd
$ ls -l /etc/passwd
```

- ***GUID*** : il fonctionne de la même façon que le SUID, mais transposé aux membres du groupe.
  - symboliquement s à la place du x du groupe g (octale 2000)

```
$ sudo chmod g+s /etc/passwd
```

Droits étendus : SUID, GUID, Sticky Bit

- ***Le "sticky bit"*** : traduit en "bit collant", réglemente le droit w sur le répertoire, en interdisant à un utilisateur quelconque de supprimer un fichier dont il n'est pas le propriétaire
  - noté t à la place du droit x sur la catégorie other du répertoire,
  - ne supprime pas le droit d'accès x (s'il est accordé).
  - si le droit x n'est pas accordé à la catégorie other, à la place de t c'est la lettre T qui apparaitra
  - Valeur octale 1000

    Ex:

    `$ chmod +t rep`

    `$ --> d ... ... ..t rep =>` si le répertoire a le droit x

    `$ --> d ... ... ..T rep =>` sinon

Outils de gestion de l'arborescence

Commande	Description
pwd	affiche le chemin du répertoire courant
cd	change le répertoire courant (commande interne du shell)
basename	extrait le nom du fichier d'un chemin complet
chmod	modifie les droits d'un fichier
chgrp	change le groupe propriétaire du fichier
chown	change l'utilisateur propriétaire du fichier
cp	copie de fichiers
ls	affiche la liste des fichiers d'un répertoire
mkdir	crée des répertoires
mknod	crée un nom de fichier temporaire unique
rm	détruit des fichiers
rmdir	détruit des répertoires
mv	déplace (ou renomme) des fichiers
touch	met à jour les dates d'accès des fichiers
df	affiche la place disque disponible
du	donne la place disque utilisée par le répertoire courant
file	donne le type de fichier
mtools	ensemble d'outils pour la gestion des disquettes MS-DOS
mdir	affiche la liste des fichiers d'une disquette MS-DOS

Visualisation et édition de texte

Commande	Description
cat	concatène les fichiers sur la sortie standard
sort	trie les lignes de texte en entrée
sed	effectue des modifications sur les lignes de texte
more	affiche à l'écran l'entrée standard en mode page par page
less	affiche à l'écran l'entrée standard en mode page par page avec des possibilités de retour en arrière
cut	permet d'isoler des colonnes dans un fichier
expand	transforme les tabulations en espaces
head	affiche les n premières lignes
join	permet de joindre les lignes de deux fichiers en fonction d'un champ commun
paste	concatène les lignes des fichiers
tail	affiche les n dernières lignes du fichier
tac	concatène les fichiers en inversant l'ordre des lignes
uniq	élimine les doublons d'un fichier trié
rev	inverse l'ordre des lignes d'un fichier
pr	formate un fichier pour l'impression
tee	écrit l'entrée standard sur la sortie standard et dans un fichier
tr	remplace ou efface des caractères

Travailler avec l'éditeur de texte : vi

- **vi**, est un éditeur de texte en mode écran qu'il faut absolument connaître en tant qu'administrateur car c'est souvent le seul disponible sur les machines.
- Commande : `$ vi <fichier>`

  Ex : `$ vi test.sh`
- L'éditeur fonctionne avec deux modes principaux :
  - un mode commande pour se déplacer dans le texte.
  - un mode saisie de texte dont on sort en appuyant sur <ESC>.

Travailler avec l'éditeur de texte : vi

- Les commandes de déplacement sur le texte :

Commande	Fonction
h	déplace le curseur d'un caractère vers la gauche
l	déplace le curseur d'un caractère vers la droite
k	déplace le curseur d'une ligne vers le haut
j	déplace le curseur d'une ligne vers le bas
w	déplace le curseur au début du mot suivant
b	déplace le curseur au début du mot précédent
W	déplace le curseur au début du mot suivant séparé par des blancs
B	déplace le curseur au début du mot précédent séparé par des blancs
e	déplace le curseur à la fin du mot
E	déplace le curseur à la fin du mot séparé par des blancs
<CTRL>-f	affiche la page suivante
<CTRL>-b	affiche la page précédente
<CTRL>-d	affiche la demi-page suivante
<CTRL>-u	affiche la demi-page précédente

Travailler avec l'éditeur de texte : vi

- commandes pour saisir et modifier du texte

Commande	Fonction
I	passe en mode insertion avant le caractère courant
A	passe en mode ajout après le caractère courant
o	ajoute une ligne après la ligne courante et passe en insertion en début de ligne
O	ajoute une ligne avant la ligne courante et passe en insertion en début de ligne

- commandes pour saisir et modifier du texte

Commande	Fonction
x	détruit le caractère courant
dd	détruit la ligne courante
y	place le caractère courant dans le tampon
yy ou Y	copie la ligne entière dans le tampon
P	copie le contenu du tampon avant le curseur
p	copie le contenu du tampon après le curseur
u	annule la dernière modification apportée au texte

Travailler avec l'éditeur de texte : vi

- Les commandes de manipulation de fichiers

Commande	Fonction
:w	écrit dans le fichier en cours
:w nom	écrit dans le fichier `` nom ''
:q	quitte l'éditeur
:wq	sauve et quitte
:e nom	lit le fichier `` nom ''
:e #	lit le fichier précédent
:n	lit le fichier suivant sur la ligne de commande

Travailler avec les shells

- ***Le shell :***
  - interpréteur de commandes, est un programme qui sert d'intermédiaire entre l'utilisateur et le système d'exploitation.
  - Sa tâche essentielle est l'exécution de programmes
  - Pour cela, il effectue (en boucle infinie) :
    - la lecture d'une ligne
    - sa compréhension comme une demande d'exécution d'un programme avec d'éventuels paramètres.
    - le lancement de ce programme avec passage des paramètres d'éventuelles redirections d'entrées-sorties
    - les exécutions des scripts (fichiers de commandes)

Travailler avec les shells

- ***Demarrage du shell :***
  - Lors de la création de son compte, un utilisateur est associé à un type de shell
  - Lire le fichier /etc/passwd : le dernier champ contient le nom du fichier exécutable (shell par défaut) /bin/bash
  - Le shell associé est ainsi lancé automatiquement dès la saisie du login utilisateur.
  - Il poursuit sa configuration en exécutant des scripts globaux à tous les utilisateurs et des scripts liés au compte et qui permettent une personnalisation.
  - Enfin, il affiche le prompt et se met en attente de la lecture d'une commande.
  - Jusqu'à la commande exit, pour quitter le shell (ce qui équivaut à se déconnecter (logout))

Travailler avec les shells

- ***Personnalisation des commandes bash :***
  - Pour créer des alias
  - Editer le fichier : ~/.bashrc (/home/rakoto/.bashrc)

```
sudo vi ~/.bashrc

alias lll='ll | less'

alias avoahy='ls -al'

alias izaho='whoami'
```

  - Esc + :wq pour quitter vi et enregistrer les modifications
  - Pour un alias global pour tout les utilisateur fichier /etc/bashrc

Variables d'environnement

- Variable d'environnement :
  - les variables d'environnement, sont des paramètres qui permettent de modifier le comportement et l'exécution des programmes
  - Il `est d'usage que les noms des variables d'environnement soient écrits complètement en majuscules
  - Comme tout programme, le shell dispose d'un environnement, qu'il utilise pour stocker ses propres variables
  - définir une variable du shell : `$ VARIABLE=valeur`
    - Ex : `$ BONJOUR="Bonjour le monde \!"`
    - Afficher le contenu : `$ echo $BONJOUR`
  - Pour le rendre accessible par d'autre programme : `$ export BONJOUR`
  - Pour supprimer : `$ unset BONJOUR`

Variables d'environnement

- Variable d'environnement usuelles :
- HOME : Chemin du répertoire personnel de l'utilisateur.
- USER : Nom de login de l'utilisateur, également dans variable d'environnement LOGNAME.
- SHELL : Chemin sur le fichier de programme du shell actuellement utilisé. Sous Linux, il s'agit souvent du shell bash.
- PATH : Liste des répertoires dans lesquels les programmes à exécuter seront recherchés.
- LD_LIBRARY_PATH : Liste des répertoires dans lesquels les librairies dynamiques seront recherchées
- ...

Programmation shell de base

- Caractère d'échappement et chaînes de caractères :
  - certain caractères sont interprétés d'une manière spéciale (>, <, |, !, $, =, \, ...)
  - Pour les utiliser, il suffit de faire précéder ces caractères du caractère d'échappement antislash (\)
  - pour créer un répertoire nommé < : `$ mkdir \<`
  - Chaine de caratère, on utilise le des guillemets simple ('... ')
- Les substitutions
  - Variables : Si on veut utiliser des variables dans une chaine de caratère, on utilise le guillemet double
    - Ex : $ echo “mon nom est $USER”
  - Utilisateur : le caractère tilde ('~') remplace le chemin sur le répertoire personnel de l'utilisateur courant
  - Ex : `echo ~` => `echo “/home/rakoto”` pour l'utilisateur rakoto

Programmation Shell de base

- ne surtout pas confondre les guillemets simples (caractère ') avec les guillemets inverses (caractère `)
  - Le premier (') sert à définir des chaînes de caractères,
  - le deuxième (`) sert exécuter une commande et à en inclure le résultat dans une autre commande
- Les guillemets simples ne permettent pas de bénéficier des fonctionnalités de substitutions du Shell,(remplacement d'une variable par sa valeur)
- Pour cela, on utilise le guillemet double (“)
  - Exemple :

```
echo ”Mon nom est $USER”
```

=> affiche “`mon nom est Rakoto`” si Rakoto est l'utilisateur actuel (dans $USER)

Par contre :

`echo 'Mon nom est $USER'` => affiche ”`mon nom est $USER`”